

10. Tipe Data List

10.1 Tujuan Praktikum

Setelah mempelajari Bab ini, mahasiswa diharapkan dapat:

1. Dapat memahami sifat-sifat list dan dapat mendefinisikan list.
2. Dapat memahami dan melakukan operasi-operasi CRUD (Create, Read, Update dan Delete) pada list.
3. Dapat memahami dan menggunakan fungsi-fungsi Python yang berkaitan dengan list.
4. Dapat memahami dan menggunakan list sebagai parameter fungsi.

10.2 Alat dan Bahan

Praktikum ini membutuhkan perangkat komputer yang memiliki spesifikasi minimum sebagai berikut:

1. Terkoneksi ke Internet dan dapat mengunduh package-package Python.
2. Mampu menjalankan sistem operasi Windows 10 atau Ubuntu Linux.

Perangkat lunak yang diperlukan untuk mendukung praktikum ini adalah sebagai berikut:

1. Python 3.7 atau 3.8 yang terinstall menggunakan Anaconda atau Installer Python lainnya.
2. Web Browser (Mozilla Firefox, Microsoft Edge atau Google Chrome).
3. Command Prompt (jika menggunakan Windows).
4. Terminal (jika menggunakan Linux).
5. Editor Python (Visual Studio Code, PyCharm, Spyder atau editor-editor lainnya yang mendukung Python).

10.3 Materi

10.3.1 Sifat-sifat List

List pada Python adalah rangkaian nilai-nilai yang dapat diakses menggunakan satu nama tunggal. Apa bedanya dengan String? String adalah rangkaian dari karakter-karakter, sedangkan

list dapat berisi karakter, integer, float maupun tipe data lainnya. List juga bisa berisi list lainnya. Rangkaian nilai-nilai tersebut dituliskan di dalam [] seperti contoh berikut ini:

```
nilai_ujian = [80,75,70,90,81,84,92,71,65,80,70]
nama_pahlawan = ['Sukarno', 'Diponegoro', 'Jend. Sudirman', 'Cut Nya Dhien']
nilai_campuran = ['Javascript', 20, 34.4, True]
list_dalam_list = [23, [22, 20], 45]
```

List juga memiliki perbedaan lain dengan String, yaitu list bersifat **mutable**, sedangkan String bersifat **immutable**. Mutable artinya nilainya dapat diubah secara langsung, seperti yang ditunjukkan pada kode program berikut ini:

```
# definisikan list berisi 4 nilai
data = [10,20,30,40]
# ubah nilai index ke-0 menjadi 50
data[0] = 50
# isinya sekarang: 50, 20, 30, 40
print(data)
```

```
# definisikan sebuah string
nama = 'Antonius Rachmat'
# ubah karakter index ke-0 menjadi Z
nama[0] = 'Z'
# tidak akan mencapai baris ini karena muncul error
# TypeError: 'str' object does not support item assignment
print(nama)
```

Perbedaan berikutnya, jika ada dua string yang isinya sama, keduanya menunjuk (merujuk/mengacu) pada object yang sama. Sedangkan pada list jika ada dua list dengan isi yang sama, keduanya menunjuk pada object yang berbeda. Hal tersebut dapat ditunjukkan dengan Python shell berikut ini:

```
>>> a = 'banana'
>>> b = 'banana'
>>> a is b
True

>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
```

10.3.2 Mengakses dan Mengubah isi List

Cara mengakses elemen list adalah dengan menggunakan indeksinya. Kita dapat mengubah isi dari list dengan menggunakan indeksinya juga. Contoh:

```
thislist = ["apple", "banana", "cherry"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])

thislist[1] = "jeruk"

print(thislist)

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist2[1:3] = ["blackcurrant", "watermelon"]
print(thislist2)
```

10.3.3 Fungsi-fungsi Untuk List

Penambahan list juga bisa dilakukan dengan menggunakan operator +, pengulangan menggunakan *. Contoh:

```
thislist = [1,2,3,4,5]
thislist2 = [6,7,8]

listbaru = thislist + thislist2
listbaru2 = thislist * 2

print(thislist)
print(thislist2)
print(listbaru)
print(listbaru2)
```

Python memberikan beberapa method tentang list:

1. **append**. Metode yang digunakan untuk menambahkan elemen baru dan dianggap sebagai kesatuan objek pada bagian akhir list.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.append(['tejo', 'ujo'])
```

2. **extend**. Metode yang digunakan untuk menambahkan elemen pada sebuah list, dan memper-lakukan setiap element baru sebagai elemen list secara individual.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.extend(['tejo', 'ujo'])
```

3. **sort**. Metode yang digunakan untuk mengurutkan elemen pada sebuah list.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.sort()
```

4. **pop**. Metode yang digunakan untuk menghapus elemen pada sebuah list. Jika indeks elemen sudah diketahui dan ingin mendapatkan nilai yang dihapus.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.pop(1) #mengembalikan 'felix'
```

5. **del.** Metode untuk menghapus elemen list tapi tidak perlu menampilkan elemen yang dihapus.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> del nama[2]
```

6. **remove.** Metode untuk menghapus elemen list berdasarkan nilai tertentu.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.remove('kuncoro')
```

7. **reverse.** Metode untuk membalik urutan list.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.reverse()
```

Selain itu terdapat beberapa metode built-in yang dapat diakses oleh list, yaitu:

1. **min():** untuk mengetahui nilai minimal dari elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> min(nama)
```

2. **max():** untuk mengetahui nilai maksimal dari elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> max(nama)
```

3. **sum():** untuk mengetahui jumlah dari total elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> sum(nama)
```

4. **len():** untuk mengetahui panjang list. Contoh:

```
>>> nama = [5,3,90,1]
>>> len(nama)
```

List juga dapat dilakukan list comprehension. Dengan sintaks ini operasi pada list jadi lebih singkat. Contoh berikut adalah list comprehension untuk memeriksa apakah dalam elemen list mengandung substring "or".

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if "or" in x]
```

Contoh untuk mengubah semua elemen list menjadi huruf besar.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x.upper() for x in nama]
```

Contoh untuk memeriksa mengambil elemen list yang panjangnya lebih dari 6 huruf.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if len(x) > 6]
```

10.3.4 List Sebagai Parameter Fungsi

Jika list digunakan sebagai parameter fungsi maka memiliki sifat yang juga mutable. Apa yang anda ubah di dalam fungsi akan mengubah nilai aslinya. Contoh:

```
>>> def ubah(data):
...     data[0] = "anton"
...
>>> data = ["a", "b", "c"]
>>> ubah(data)
>>> data
['anton', 'b', 'c']
```

Keuntungan Menggunakan List sebagai Parameter:

1. Fleksibilitas: fungsi dapat memproses berbagai jenis data dengan menggunakan list.
2. Reusabilitas: fungsi dapat digunakan kembali dengan list yang berbeda.
3. Pembagian Tugas: list dapat membantu membagi tugas menjadi bagian-bagian yang lebih kecil.
4. Pengorganisasian Data: list membantu mengorganisir data sebelum diproses oleh fungsi.

Contoh lain untuk menghapus elemen pertama list:

```
>>> def hapus(data):
...     return data[1:]
...
>>> hapus(data)
['b', 'c']
>>> data
['anton', 'b', 'c']
>>> data = hapus(data)
>>> data
['b', 'c']
```

10.4 Kegiatan Praktikum

Kasus 1.

Buatlah fungsi yang dapat mengembalikan nilai True jika dari dua list yang diinputkan memiliki minimal 1 nilai yang sama.

Jawab:

```
def ada_elemen_sama(list1, list2):
    return any(elemen in list2 for elemen in list1)

list_a = [1, 2, 3, 4]
list_b = [5, 6, 3, 8]

hasil = ada_elemen_sama(list_a, list_b)
print(hasil) # Output: True
```

Kasus 2.

Buatlah sebuah fungsi untuk mencari bilangan terbesar kedua dari sebuah list yang diinputkan.

Jawab:

```
def cari_terbesar_kedua(data):
    if len(data) < 2:
        raise ValueError("List harus memiliki minimal dua elemen.")

    unik = list(set(data)) # Menghapus duplikat
```

```

    if len(unik) < 2:
        raise ValueError("List tidak memiliki cukup bilangan unik.")

    unik.sort(reverse=True)
    return unik[1]

angka = [10, 5, 8, 20, 20, 4]
hasil = cari_terbesar_kedua(angka)
print(hasil) # Output: 10

```

Kasus 3.

Buatlah fungsi untuk sebuah list merupakan subset dari list lainnya. Input dari fungsi adalah 2 buah list, list A dan B. Fungsi akan memeriksa apakah list A merupakan subset dari A atau bukan.

Jawab:

```

def apakah_subset(list_A, list_B):
    return set(list_A).issubset(set(list_B))

A = [1, 2, 3]
B = [0, 1, 2, 3, 4, 5]

hasil = apakah_subset(A, B)
print(hasil) # Output: True

```

10.5 Latihan Mandiri

Latihan 10.1 Buatlah fungsi untuk mencari 3 bilangan terbaik dari sebuah list. Bilangan terbaik adalah bilangan dimulai dari yang paling tinggi. ■

Latihan 10.2 Buatlah program yang dapat menerima input bilangan-bilangan dari pengguna dan kemudian jika pengguna memasukan kata "done" maka program akan menampilkan bilangan rata-ratanya. ■

Latihan 10.3 Buatlah program yang menerima sebuah file teks yang berisi sebuah artikel atau berita. Kemudian program akan memeriksa semua kata yang ada di dalam artikel teks tersebut, kemudian program akan menampilkan semua kata yang unik dari artikel tersebut ■